2026-4-28 C++的一些前导知识

- 1.缺省参数
  - 1.1 什么是缺省参数?
  - 1.2缺省参数的特性
  - 1.3缺省参数的应用场景
- 2.函数重载
  - 2.1什么是函数重载
  - 2.2 函数重载的类型
  - 2.3有关函数重载的易错点分析
- 3.引用
  - 3.1什么是引用
  - 3.2 引用的特性
  - 3.3 引用的应用场景
  - 3.4 const引用
  - 3.5 引用与指针的关系

1.缺省参数

1.1 什么是缺省参数?

缺省参数是声明或者定义函数时为函数的参数指定一个缺省值。在调用该函数时，如果没有指定实参则采用该形参的缺省值，否则则使用指定的实参，缺省参数分为全缺省和半缺省参数。（有的地方把缺省参数也叫默认参数）

1.2缺省参数的特性

1. 全缺省：全部形参给缺省值
2. 半缺省：部分形参给缺省值
3. 函数的定义和声明分开时，缺省参数不能在声明和定义两头同时给缺省值，规定只能在函数声明的地方给缺省值。
   C++规定传参时，实参必须从左到右依次匹配形参，不能跳跃传参。

```
void Func1(int x, int y, int z) {
    cout << x << y << z << endl;
}
void Func2(int x=1, int y=1, int z=1) {
    cout << x << y << z << endl;//全缺省
```

```
}
void Func3(int x , int y=1, int z = 1) {
    cout << x << y << z << endl;//半缺省
}
int main() {
    Func1(1, 1, 1);
    Func2();
    Func3(2, 2, 2);
    return 0;
}
```

1.3缺省参数的应用场景

下面先列举一个常见的运用场景：初始化数组或者申请内存的时候，大多使用缺省参数，少数情况需要自己定义。

```
typedef struct {
    DataType* arr;
    int size;
    int capacity;
}SeqList;
void  IniteSqeList(SeqList* list,DataType capacity = 4) {
    //这里使用缺省值就避免了给capacity传参
    //规则一旦一个参数有缺省值，那么它右边的参数都有缺省值
    DataType* newList = (DataType*)malloc(capacity * sizeof(DataType));
    if (newList == NULL) {
        perror("error");
        exit(1);
    }
    list->arr = newList;
    list->capacity = capacity;
    list->size = 0;

}
int main(){
    SeqList Sq;
    IniteSqeList(&Sq);//这样可以让顺序表有一个初始的大小
    //IniteSqeList(&Sq, 100);//如果你已经知道了这个顺序表需要多大
    //那就可以一次开好避免重复扩容造成的时间消耗
    free(Sq.arr);
    return 0;
}
```

2.函数重载

2.1什么是函数重载

在C++中支持创建函数的时候可以用同一个函数名，但是函数参数的个数、类型、顺序不同，这样的函数可以称为函数的重载（这种C语言是不支持的）。我们可以说这个就是一个函数的多态表达形式。

2.2 函数重载的类型

假设我们有下述函数：

```
void Swap(int x, int y) {
    int tmp;
    tmp = x;
    x = y;
    y = tmp;
    cout << x << y << endl;
}
```

类型不同的函数重载(下述二者都是类型不同的重载)：

```
void Swap(char x, char y) {//类型不同
    char tmp;
    tmp = x;
    x = y;
    y = tmp;
    cout << x << y << endl;
}
void Swap(int x, float y) {//类型不同
    int tmp;
    tmp = x;
    x = y;
    y = tmp;
    cout << x << y << endl;
}
```

顺序不同的重载：

```
// 正确的顺序不同构成重载的例子
void Print(int a, double b) {
    cout << "int, double" << endl;
}

void Print(double a, int b) {
    cout << "double, int" << endl;
}

int main() {
    Print(1, 2.2);   // 调用第一个
    Print(2.2, 1);   // 调用第二个
    return 0;
}out << x << y << endl;
```

2.3有关函数重载的易错点分析

1. 函数的返回值不同能不能说这两个函数构成重载：答案肯定是不能的，因为返回值可以不接收，你没有办法去区分。
2. 分析上述的代码会存在什么样的问题?

```
void Func() {
    int a = 1;
    cout << a << endl;
}
void Func(int a=1) {
    cout << a << endl;
}
int main() {
    Func();
    return 0;
}
```

上述代码是不对的，为什么呢？因为我们发现上面的两个Func( )函数都不需要传递参数就可以直接调用，也就意味着你在主函数里面调用Func( )的时候编译器好像使用哪个都是对的，这个时候就会编译错误。

3.引用

3.1什么是引用

引用就是给已经存在的变量取一个别名（这里不是定义一个新的变量），编译器不会为引用开辟一块新的内存空间。这里就像你自己的小名，朋友给你的昵称一样，实际上说的都是都是一个东西。

类型& 引用别名 = 引用对象

C++中为了避免引入更多的符号，就会对C语言中的符号进行一些复用。引用实际上和取地址使用的就是一个符号&。我们需要从使用方法的角度进行区分。

3.2 引用的特性

1. 引用在定义的时候必须初始化
2. 一个变量可以有多个引用
3. 引用一旦引用了一个实体，就不能再引用一个实体。

3.3 引用的应用场景

1. 引用做函数形参，修改形参影响实参

```
void Swap(int& a, int& b) {
    int tmp = a;
    a = b;
    b = tmp;
}

int main() {
    int x = 1, y = 2;
    cout << x <<" "<< y << endl;
    Swap(x, y);//这里可以避免传址调用
    cout << x <<" "<< y << endl;
    return 0;
}
```

2. 做函数形参，减少拷贝提高效率

```
//如果你现在想传递如果你现在想传递一个占用内存很大的结构体或对象
//我们知道形参实际上是实参的一个拷贝，当形参的容量非常大的时候效率回很低，例如：
struct Student {
    char name[100];
    int scores[100];
};
//void StudentPrint(const Student& s) {//传引用传参实际指向的都是同一片地址
```

```
//  cout << s.name << endl;
//}
void StudentPrint(const Student s) {//这样的话形参会将实参一整个的拷贝过来
    cout << s.name << endl;
}
int main() {
    Student st;
    StudentPrint(st);//让s直接作为st的别名，指向同一块地址使用
    //原理就是
    int a = 5;
    int& b = a;
    return 0;
}
```

3. 引用做返回值类型，修改返回对象

```
//我们可以以学生管理系统的成绩存储为例举一个小例子
void  IniteSqeList(SeqList& list,DataType capacity = 4) {
    //这里使用缺省值就避免了给capacity传参
    //规则一旦一个参数有缺省值，那么它右边的参数都有缺省值
    DataType* newList = (DataType*)malloc(capacity * sizeof(DataType));
    if (newList == NULL) {
        perror("error");
        exit(1);
    }
    list.arr = newList;
    list.capacity = capacity;
    list.size = 0;
}
void PushSqeList(SeqList& list,DataType x){
    if (list.size == list.capacity) {
        int Newcapacity = list.capacity * 2;
        DataType* newSeqList= (DataType*)realloc(list.arr, sizeof(DataType)
* Newcapacity);
        if (list.arr == NULL) {
            perror("error");
            exit(1);
        }
        Newcapacity = list.capacity;
        list.arr = newSeqList;

    }
    list.arr[list.size++] = x;
}
DataType& At(SeqList& list, int pos) {
```

```
    return list.arr[pos];
}
int main(){
    SeqList scorea;
    IniteSqeList(scorea);
    //存放学生成绩
    PushSqeList(scorea, 0);//避开下标0
    PushSqeList(scorea, 99);
    PushSqeList(scorea, 89);
    PushSqeList(scorea, 95);
    PushSqeList(scorea, 94);
    cout << scorea.arr[2] << endl;
    At(scorea, 2)=100;//返回的是下标为2的学生成绩的别名
    //直接指向下标为2的学生成绩的空间并修改成绩为100
    cout << scorea.arr[2] << endl;

    free(scorea.arr);
    return 0;
}
```

4. 引用做返回值类型，减少拷贝，提高效率

```
//这个我们可以举个例子来解释
#include<iostream>
using namespace std;
struct Cake {
    int weight;//蛋糕的重量
    string flavor;//蛋糕口味
    int layer[10];//十层大蛋糕
};

void Init(Cake& cake){
    cake.flavor = "巧克力";
    cake.weight = 10;
}

Cake fridge;

Cake getCakeCopy() {
    return fridge;//返回的是这个蛋糕的一个拷贝
}
Cake& getCakeRef() {
    return fridge;//这个东西是告诉你这块蛋糕在哪里？
}
```

```
int main() {
    Init(fridge);
    Cake myCake1= getCakeCopy();
    cout << "myCake1原本的重量:"<<myCake1.weight << endl;
    myCake1.weight = 500;//改的是复制品（改的不是全局的哪个，即不是蛋糕本身），冰箱里的蛋糕没变
    cout << "myCake1:"<<myCake1.weight << endl;
    cout << "fridge:" << fridge.weight << endl;

    Cake& myCake2 = getCakeRef();
    cout <<"myCake2原本的质量:"<< myCake2.weight << endl;
    myCake2.weight = 300;//改的直接是冰箱里的蛋糕，冰箱里的蛋糕没变
    cout <<"myCake2:"<<myCake2.weight << endl;
    cout << "fridge:"<<fridge.weight << endl;
    return 0;
}
```

总结：
上述例子可以说明，从角色上可以作为函数的形参或者返回值，从功能上一般是用于减少拷贝或者参数修改。

用法	核心目的	是否减少拷贝	是否修改原对象
1. 做函数形参，修改形参影响实参	替代指针，简化语法	否	是
2. 做函数形参，减少拷贝	避免大对象拷贝	是	否（加const）
3. 做返回值，修改返回对象	容器下标式访问	是	是
4. 做返回值，减少拷贝	避免大对象返回拷贝	是	否（加const）

3.4 const引用

1. 可以对const对象进行引用，但是引用的类型前面必须加上const。但是反过来，可以引用类型前加const可以用来引用一个普通的变量。（权限可以缩小，但是权限不能放大）

```
int main() {
    //可以对const对象进行引用
    const int x = 10;
    //但是引用的类型前面必须加上const
    //int& a = x;这样是不可以的，你想一个变量做它自己的时候是不可以改变的
    // 咋一给它取个外号就可以改变了？这明显不合理，这个变量本身的使用权限被放大了
    const int& a = x;//必须const修饰的变量，这样才是正确的
```

```
    //可以引用类型前加const可以用来引用一个普通的变量
    int y = 20;
    const int& b = y;
    //但是原本如果不可以修改，取一个不可以被修改的别名这样就是正确的了

    return 0;
}
```

2. 有一些需要注意的场景，如果需要引用对象的过程中如果会经过一个临时对象，C++规定临时对象具有常性，这里就会隐含的触发权限放大导致编译不通过，所以必须常引用才可以。

```
//权限放大:实际本质上就是让原本不能被修改的变量变得可以修改
int main() {
    int x = 10;
    //场景1，引用一个表达式
    //int& rb = x * 3;这种情况下，x * 3的结果会保存在一个临时对象中

    //场景2，强制类型转换
    double y = 1.11;
    //int& rb = y;类型转换的过程中也会产生临时对象存储中间值
    return 0;
}
```

临时对象：编译器需要一个空间暂存表达式的求值结果时临时创建一个未命名的对象，在C++中，把这个未命名对象叫做临时对象。

3.5 引用与指针的关系

在C++中指针和引用二者的功能具有重叠性，二者相辅相成，但是各自又有各自的特点，都是无法替代的。

1. 在语法概念上引用时给变量取别名不开空间，指针是存储一个变量的地址，需要开空间。
2. 引用在使用时必须初始化，而指针建议初始化，在语法上指针是否初始化不是必须的。

```
int main() {
    //引用的使用必须初始化
    int x;
    //int x = 10;
```

```
    int& a = x;

    //指针的使用没有必须说明要初始化
    int* b;
    return 0;
}
```

3. 一个变量在称为一个对象的引用之后就不能再成为别的对象的引用了，而指针是可以不断改变指向的。

```
int main() {
    int x = 10;
    int y = 10;
    int& a = x;
    int& a = y;//a不能同时作为x和y的别名

    int* p = &x;
    p = &y;//指针的指向是可以改变的
    return 0;
}
```

4. 引用可以直接访问被引用的对象，而指针访问被指向的对象时需要解引用。
5. 引用和指针在sizeof中的意义不同，sizeof求引用的结果是这个对象类型的大小，但指针始终是地址空间所占的字节数（32：4、64：8）

```
int main() {
    char x = 'a';
    char y = 'a';
    char& a = x;
    char* p = &x;
    cout << sizeof(a) << endl;// 1
    cout << sizeof(p) << endl;// 8（64位）
    return 0;
}
```

6. 指针容易出现空指针和野指针的情况，但是引用不会，引用要相对安全一些。

维度	引用	指针
本质	变量的别名（不占用额外空间）	存储变量的地址（占用空间）
语法概念	不开空间	需要开空间（4 / 8 字节）
初始化要求	**必须**初始化	建议初始化（语法上非必须）
是否可重新绑定	不可改变指向	可以随时改变指向
访问原对象	直接访问	需要解引用（`*p`）
`sizeof` **结果**	对象本身的大小	指针本身的大小（4 / 8 字节）
空值 / 野值风险	相对安全，不指向空	容易出现空指针、野指针
典型应用场景	函数参数、返回值、运算符重载	动态内存管理、链表、数组遍历

- 为什么说引用不额外占用空间?
  因为引用不是对象，不占用对象所在的存储区域（栈 / 堆 / 静态区）。它是被引用对象的别名，语法上不保证独立存储。
- 引用既然不占存储，计算机怎么记住它?
  编译期间通过符号表记住绑定关系；运行期间通常用一个常量指针（占空间）来实现引用语义。这个底层指针虽然占空间，但**不属于“被引用对象的存储”**，因此语法概念上仍然说引用不额外占用空间。

```
int a = 10;       // a 在栈上 / 静态区 / 作为对象存在
int& r = a;       // r 不独立占用对象存储,也就是不占用对象的存储区域
int* p = &a;      // p 是一个独立的对象（占栈空间）
```

在大多数编译器的底层实现中，引用是通过常量指针来实现的。但在语法概念上，引用和指针是完全不同的。

```
int& ref = a;
// 底层 roughly 等价于
int* const ref = &a;
```